



PSTA - Plataforma de Seguridad Transaccional Adaptativa

Entrega ejecutiva - Prueba técnica Líder Backend - Gestión de fraude - Autenticación – Alejandro Roldán Pineda

1. Resumen ejecutivo

La propuesta parte de una lectura concreta del reto: el problema no es únicamente la caída de un factor de autenticación ni la detección tardía de fraude, sino la desconexión entre dos dominios que deberían operar juntos: **riesgo y autenticación**. La propuesta no intenta retar más transacciones; intenta aplicar la fricción correcta en el momento correcto.

La PSTA se plantea como una plataforma backend donde el riesgo transaccional se calcula en tiempo real, la política de autenticación traduce ese riesgo en una acción concreta, los factores de autenticación se ejecutan con fallback controlado, la auditoría conserva evidencia de cada decisión y la respuesta antifraude actúa cuando el patrón supera el umbral permitido.

Esta entrega no busca presentar una plataforma bancaria completa. Su objetivo es mostrar cómo abordaría el problema como líder técnico: entender la causa raíz, separar dominios, definir contratos, validar los escenarios críticos, preparar la operación y dejar una base que el equipo pueda construir, sostener y evolucionar.

El camino principal seleccionado es **Arquitectura Profunda**. Se complementa con un MVP pequeño en TypeScript, no como solución final de fraude, sino como evidencia representativa de una decisión de riesgo: señales de entrada, score, nivel de riesgo, reason codes, acción requerida y evento de auditoría.

Cómo leer esta entrega

- **PDF:** historia ejecutiva, estrategia técnica y criterios de liderazgo.
- **README:** mapa del repositorio e índice de entregables.
- **docs:** profundidad técnica por tema.
- **diagrams:** arquitectura, componentes y secuencias.
- **assets/diagrams:** diagramas exportados para lectura rápida.
- **mvp:** implementación representativa del servicio de decisión de riesgo.

Repositorio

https://github.com/alejoroldan608/psta_technical_challenge

2. Entendimiento del problema

El reto no se debe leer como dos fallas aisladas: una caída de Clave Dinámica y una detección tardía de fraude. Ambos incidentes muestran un problema más profundo: los dominios de autenticación y riesgo existen, pero no toman decisiones coordinadas en el momento en que ocurre la transacción.

- a. En el primer incidente, el segundo factor se saturó y no existía un mecanismo automático de fallback gobernado por riesgo. El sistema trataba a todos los clientes de forma parecida, sin distinguir si la operación era de bajo, medio o alto riesgo. Eso terminó afectando a clientes legítimos y generando abandono de transacciones.

- b. En el segundo incidente, el fraude por velocidad no fue detectado a tiempo. El sistema observaba el comportamiento, pero no reaccionaba con la velocidad necesaria para bloquear, retener o endurecer la autenticación antes de que el daño creciera.

La causa raíz no es únicamente tecnológica. También es de diseño de dominio y de operación. Autenticación valida factores, pero no necesariamente entiende el riesgo contextual de la transacción. Riesgos analiza patrones, pero no siempre puede intervenir la autenticación en tiempo real.

Por eso, la solución no debería ser agregar otro servicio sin cambiar la forma de decidir. La PSTA debe permitir que riesgo, autenticación, auditoría y respuesta antifraude trabajen como partes de un mismo flujo de seguridad transaccional.

Desde arquitectura, el problema se puede resumir en cuatro puntos:

- Las decisiones críticas necesitan una ruta síncrona y de baja latencia.
- Los eventos son necesarios para correlación, auditoría y detección de patrones.
- La autenticación debe ser adaptativa, no igual para todas las operaciones.
- La operación debe estar preparada para fallas: saturación, caída de factores, picos de tráfico y señales de fraude.

Esto implica separar responsabilidades, pero conectar decisiones. Riesgo calcula y explica; política traduce ese riesgo en una acción; autenticación ejecuta el factor; auditoría conserva evidencia; y respuesta antifraude actúa cuando el patrón lo exige.

Objetivos de diseño:

Objetivo	Cómo se aborda en la propuesta
Reducir fricción en operaciones habituales	Autenticación silenciosa o confianza de sesión cuando el riesgo es bajo.
Endurecer operaciones inusuales	Factor fuerte cuando aparecen señales como dispositivo nuevo, ubicación inusual o monto medio/alto.
Detectar fraude por velocidad	Procesamiento de eventos en streaming y correlación por cuenta, monto, beneficiario y ventana temporal.
Mantener continuidad ante fallas	Circuit breaker y fallback según nivel de riesgo, sin aprobar operaciones críticas con factores degradados.
Garantizar trazabilidad	Auditoría de score, señales, política aplicada, factor usado, resultado y evento generado.
Evitar repetir silos	Dominios separados, contratos explícitos y ownership compartido del flujo de seguridad transaccional.

El repositorio refleja esa forma de trabajo: los documentos explican las decisiones, los diagramas hacen visible la arquitectura, los escenarios validan el diseño y el MVP aterriza una parte pequeña pero representativa de la lógica de decisión de riesgo.

3. Diseño de dominio y arquitectura lógica

La PSTA se diseña partiendo de una decisión clave: autenticación y riesgo deben seguir siendo dominios separados, pero no pueden operar aislados. Separarlos permite que cada uno evolucione con claridad; conectarlos mediante contratos y eventos permite que la decisión de seguridad sea oportuna, trazable y adaptable.

La arquitectura propuesta no busca crear muchos servicios. La división responde a responsabilidades concretas del negocio: evaluar riesgo, decidir la fricción de autenticación, ejecutar factores, auditar decisiones y activar respuesta antifraude cuando el patrón lo exige.

Dominio / componente	Responsabilidad principal
API Gateway	Recibe solicitudes de canales web, móvil y B2B. Aplica controles de entrada, trazabilidad y enrutamiento.
Transaction Security Service	Orquesta la decisión antes de ejecutar una transacción: permitir, retar, retener o bloquear.
Risk Decision Service	Calcula score, nivel de riesgo y reason codes usando señales como monto, dispositivo, ubicación, sesión y actividad reciente.
Authentication Policy Service	Traduce el nivel de riesgo en una acción concreta: autenticación silenciosa, factor fuerte, retención o bloqueo.
Factor Orchestrator	Ejecuta Clave Dinámica, push, biometría u otro factor disponible. También aplica fallback cuando un factor falla.
Streaming Risk Processor	Correlaciona eventos para detectar patrones acumulados, como ataque de velocidad o múltiples beneficiarios nuevos.
Fraud Response Service	Aplica bloqueos, retenciones o generación de casos cuando el riesgo supera el umbral permitido.
Audit Service	Registra señales, score, política aplicada, factor usado, resultado y eventos generados.

El flujo crítico se resuelve de forma síncrona: el cliente no puede esperar un proceso batch para saber si su operación continúa. Por eso, la evaluación de riesgo y la política de autenticación deben responder dentro del flujo transaccional.

Los eventos quedan para lo que no debe bloquear la experiencia inmediata: correlación, auditoría, observabilidad y aprendizaje operacional. Así se cubren los dos ritmos del problema: la decisión rápida de una transacción y la detección progresiva de patrones. El principal trade-off es que esta separación exige contratos claros y ownership compartido. A cambio, se gana una arquitectura más entendible, operable y preparada para evolucionar sin repetir el problema original de silos entre autenticación y fraude.

4. Escenarios del reto como pruebas de arquitectura

Los cinco escenarios del reto se tratan como pruebas de la solución, no como preguntas separadas. En todos los casos se usa el mismo principio: el riesgo calcula y explica, la política decide la fricción, autenticación ejecuta el factor, eventos permiten correlación y auditoría conserva la evidencia.

Escenario	Componentes propuestos en la arquitectura	Decisión de arquitectura	Garantía del resultado
Transacción habitual de bajo riesgo	API Gateway, Transaction Security Service, Risk Decision Service, Authentication Policy Service, Audit Service	Se evalúan señales simples y de baja latencia: monto bajo, dispositivo conocido, ubicación habitual, sesión confiable y bajo volumen reciente.	El flujo se mantiene sin factor visible para no penalizar el caso mayoritario. Se audita score, señales, política aplicada y resultado.
Dispositivo desconocido con monto medio	Device Trust, Risk Decision Service, Authentication Policy Service, Factor Orchestrator, Audit Service	El dispositivo nuevo, la ubicación inusual y el monto medio elevan el score a riesgo medio.	No se bloquea de entrada; se exige factor fuerte. Si el cliente lo supera, el dispositivo queda aprendido de forma gradual y auditable.
Ataque de velocidad	Kafka/Event Broker, Streaming Risk Processor, Risk Decision Service, Fraud Response Service, Audit Service	Se correlacionan transferencias por cuenta, monto, beneficiarios nuevos y ventana temporal. El score escala con cada evento.	Antes de completar el patrón, el riesgo pasa a crítico y se bloquea temporalmente la cuenta, impidiendo nuevas operaciones mientras dure el bloqueo.
Indisponibilidad de Clave Dinámica	Factor Orchestrator, Authentication Policy Service, Circuit Breaker, Observability, Audit Service	La caída se detecta por timeouts, errores y latencia. Se abre circuito y se activa fallback según riesgo.	Bajo y medio riesgo continúan con el mejor factor disponible; alto y crítico no se aprueban con factor degradado. El retorno al factor principal es gradual.
Escalamiento de riesgo en sesión activa	Session Trust, Risk Decision Service, Authentication Policy Service, Streaming Risk Processor, Fraud Response Service	La confianza de sesión no es fija. Cada operación sensible genera eventos que recalculan riesgo dentro de la misma sesión.	Se aplica reautenticación escalonada por operación. La sesión no se invalida completa, pero una operación crítica puede ser retenida o bloqueada.

El detalle extendido de flujos, componentes y decisiones queda documentado en docs/04-flujos-escenarios.md, y los diagramas de secuencia respaldan los escenarios de bajo riesgo, caída de Clave Dinámica y ataque de velocidad.

5. Hoja de ruta de construcción por fase

No construiría la PSTA intentando resolver todo al mismo tiempo. En un sistema crítico, el orden importa. Primero se debe alinear el dominio, luego definir contratos, después construir los flujos principales y finalmente endurecer la operación. La estrategia es avanzar por fases con entregables verificables. Cada fase debe dejar una base útil para la siguiente, evitando construir componentes aislados que repitan el problema original entre autenticación y riesgo.

Fase	Objetivo	Entregable principal	Criterio de cierre
1. Alineación de dominio	Acordar lenguaje común entre riesgo, autenticación, fraude, auditoría y operación.	Mapa de dominio, señales de riesgo, matriz riesgo-factor y escenarios priorizados.	Los equipos entienden los mismos conceptos y los 5 escenarios del reto tienen flujo definido.
2. Contratos y arquitectura base	Definir cómo se comunican los dominios antes de construir.	APIs, eventos, reason codes, estructura de auditoría y diagramas C4.	Existe un contrato claro para calcular riesgo, exigir autenticación, auditar y responder ante fraude.
3. Flujo transaccional mínimo	Construir el flujo principal: evaluar riesgo, decidir factor y registrar evidencia.	Servicios base de riesgo, política de autenticación, orquestación transaccional y auditoría.	Una transacción de bajo riesgo fluye sin fricción y una de riesgo medio exige factor fuerte.
4. Riesgo en streaming y respuesta antifraude	Detectar patrones acumulados que no se ven en una sola transacción.	Procesamiento de eventos, detección de ataque de velocidad y bloqueo temporal de cuenta.	El sistema detecta el patrón antes de que se complete la séptima transacción y deja trazabilidad.
5. Resiliencia y operación controlada	Preparar la plataforma para fallas reales y operación continua.	Circuit breaker, fallback por riesgo, métricas, alertas, runbooks y pruebas de carga.	La caída de Clave Dinámica no tumba todo el flujo y las operaciones de alto riesgo no se aprueban degradadas.
6. Piloto y mejora continua	Liberar de forma gradual, medir impacto y ajustar reglas.	Despliegue canary, feature flags, tablero de monitoreo y retroalimentación con fraude/riesgo.	La solución opera con métricas claras de latencia, falsos positivos, éxito por factor y tiempo de respuesta.

Esta hoja de ruta busca reducir riesgo técnico y organizacional. No se trata solo de entregar servicios, sino de construir una plataforma que el equipo pueda entender, operar y evolucionar. Como líder técnico, priorizaría que cada fase deje evidencia: decisiones documentadas, contratos versionados, pruebas, métricas y runbooks. En un sistema de esta criticidad, el conocimiento no puede quedar en conversaciones sueltas ni depender de una sola persona.

6. Operación, resiliencia, seguridad y observabilidad

La PSTA debe diseñarse pensando desde el inicio en producción. No basta con que la arquitectura funcione en un diagrama; debe poder operar bajo picos de tráfico, fallas de dependencias, intentos de fraude y necesidad de auditoría.

En operación, la prioridad es mantener tres principios: baja latencia para el flujo crítico, degradación controlada ante fallas y trazabilidad completa de las decisiones sensibles.

Frente	Decisión estratégica
Resiliencia	Usar circuit breaker, timeouts y fallback por nivel de riesgo. Si Clave Dinámica falla, bajo y medio riesgo pueden usar factores alternos; alto y crítico no se aprueban con autenticación degradada.
Picos de tráfico	Aplicar rate limiting por cliente, canal y tipo de operación. En saturación, el sistema debe rechazar rápido antes que dejar transacciones en estado incierto.
Alta disponibilidad	Diseñar despliegue multi-zona para servicios críticos, con balanceo de tráfico y recuperación ante pérdida de nodo o zona.
Seguridad	Proteger la comunicación servicio a servicio con mTLS o tokens de servicio, cifrado en tránsito y reposo, y mínimo privilegio por componente.
Observabilidad	Medir latencia de decisión de riesgo, tasa de éxito por factor, falsos positivos, operaciones bloqueadas y tiempo entre evento de riesgo y acción tomada.
Auditoría	Registrar correlation id, señales usadas, score, política aplicada, factor ejecutado, resultado y evento generado, evitando guardar OTP, secretos, tokens o datos sensibles sin enmascarar.

El comportamiento degradado debe ser seguro. La continuidad operacional no puede significar aprobar operaciones riesgosas con menor protección. Si una dependencia falla, la política debe decidir según el nivel de riesgo: continuar, pedir factor alternativo, retener o rechazar.

También dejaría runbooks para los escenarios críticos (Complementar wiki de azure devops y documentación en Github cuando se migren los repositorios): caída de Clave Dinámica, aumento de errores por factor, crecimiento anormal de bloqueos, latencia alta del Risk Decision Service y atraso en el broker de eventos. El standby no debería improvisar durante un incidente.

La observabilidad no se limita a saber si un servicio está encendido. Debe permitir responder preguntas de negocio y operación: cuántas operaciones se aprobaron sin fricción, cuántas fueron retadas, cuántas se bloquearon, qué factor está fallando y cuánto



tiempo tarda el sistema en actuar cuando aparece una señal de riesgo (Observabilidad en dynatrace, grafana y validar si en Nebula aplicaría).

7. Estrategia de equipo y soporte continuo

La PSTA no debería construirse con equipos aislados de autenticación por un lado y fraude por otro, porque esa separación es justamente parte de la causa raíz del problema. La estrategia de equipo debe organizarse alrededor del flujo completo de seguridad transaccional: evaluar riesgo, decidir autenticación, ejecutar el factor, auditar y responder ante fraude.

Para una primera versión, conformaría un equipo justo pero con una meta clara: Ingenieros Software internos Banco enfocados en Backend, que discutamos abiertamente la problemática y pintemos arquitecturas para establecer hoja de ruta, Certificador y automatizador QA, apalancaría con una capacidad de devops y otra de devsecops Banco que nos apoyen en las prácticas de desarrollo de software, PO que nos apoye priorizando o si no hay PO, un experto de fraudes que me apoye en el conocimiento particular de fraudes. No buscaría el equipo más grande, sino uno capaz de tomar decisiones rápidas, sostener contratos entre dominios y operar la plataforma en producción.

8. MVP y alcance

El MVP incluido en el repositorio no pretende construir toda la PSTA ni resolver fraude real. Su objetivo es demostrar, con código simple y sustentable, cómo una parte crítica de la arquitectura puede aterrizarse: la decisión de riesgo de una transacción.

La implementación se enfoca en el Risk Decision Service. Recibe señales como monto, dispositivo conocido, ubicación habitual, beneficiario nuevo, confianza de sesión y número de transacciones recientes. Con esas señales calcula un score, determina un nivel de riesgo, genera reason codes, sugiere una acción de autenticación y deja un evento de auditoría.

En la arquitectura completa el Risk Decision Service sería un servicio productivo integrado con señales históricas, cache, eventos y políticas versionadas. En el repositorio implementé risk-decision-demo como una versión mínima para demostrar la lógica principal sin simular toda la plataforma.

Elemento	Qué demuestra
TypeScript + Express	Un servicio backend simple, fácil de ejecutar y revisar.
Clean Architecture simplificada	Separación entre entrada HTTP, casos de uso y dominio.
RiskScoringService	Lógica demostrativa para calcular score, nivel de riesgo y motivos.
Reason codes	Explicabilidad de la decisión, no solo un número de score.
Simulación de ataque de velocidad	Representación pequeña del escenario de múltiples transferencias en ventana corta.

Tests con Jest	Validación de bajo riesgo, riesgo medio y bloqueo por patrón crítico.
Dockerfile y docker-compose	Preparación para ejecución reproducible.

El alcance del MVP es intencionalmente acotado. No implementa una plataforma antifraude completa, pero sí demuestra la forma de llevar una decisión crítica a código: entrada clara, lógica separada por capas, resultado explicable, pruebas y documentación. La propuesta completa vive en el repositorio: arquitectura, diagramas, escenarios, decisiones y una implementación pequeña que sirve como evidencia técnica.

Como cierre, la entrega no busca vender una solución perfecta. Busca demostrar cómo abordaría el reto como líder técnico backend: entendiendo la causa raíz, tomando decisiones con trade-offs claros, preparando la operación y dejando una base que un equipo pueda construir, sostener y evolucionar.

Repositorio de entrega

https://github.com/alejoroldan608/psta_technical_challenge